

OWASPKL CTF Week 3

Malware Series - Forensic Write-Up

Date : 7 Jun 2026

Field	Value
Investigator	Mohamad Alif Ridzuan Bin Ahmad
Category	Malware Analysis / Forensics
File Given	dump.evtx (Windows Event Log)
Tool Used	Kali Linux, python-evtx, grep, curl, VirusTotal, Censys
Total Investigations	9

Background.....	3
Setup - Converting .evtx to XML.....	4
Investigation 1 - Identify the Stager File Name.....	5
Investigation 2 - Identify the Hijacked PID.....	9
Investigation 3 - Identify the Hijacked Process Name.....	10
Investigation 4 - Identify the Payload Download URL.....	11
Investigation 5 - Identify Valid C2 Commands.....	13
Investigation 6 - Identify the Payload Location.....	15
Investigation 7 - Identify the C2 Server URL.....	15
Investigation 8 - Identify the Payload Hash.....	17
Investigation 9 - Identify the Root C2 Server IP.....	18
Summary - Full Attack Chain.....	21

Background

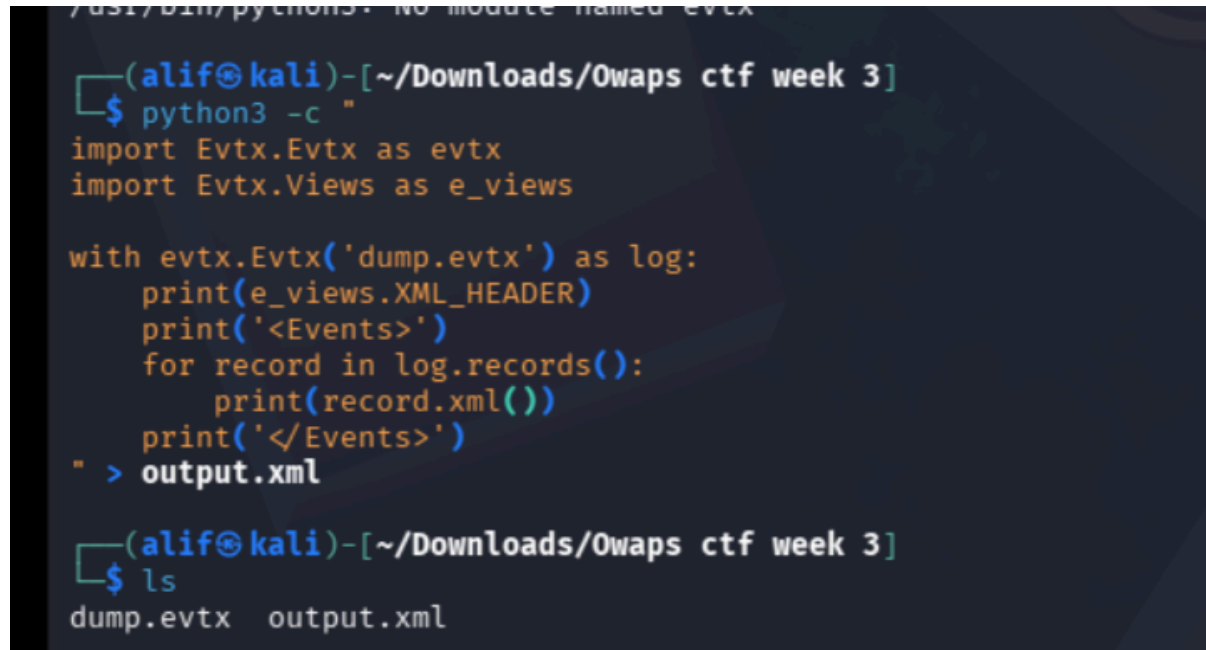
This challenge is a continuation of the Art of Evasion challenge from Week 2. It turns out the VM image contained an active malware, specifically a C2 beacon. In this series, the task is to conduct a malware analysis to figure out what happened on the victim machine.

The given file is dump.evtx, a Windows Event Log file captured from the infected machine. All investigations were conducted on Kali Linux.

Setup - Converting .evtx to XML

Windows Event Log (.evtx) files are binary and cannot be read directly. To analyze them on Kali Linux, the python-evtx library was used to convert the file into readable XML format.

Convert dump.evtx to output.xml:

A terminal window on a Kali Linux system. The prompt is (alif@kali)-[~/Downloads/Owaps ctf week 3]. The user runs \$ python3 -c "import Evtx.Evtx as evt; import Evtx.Views as e_views; with evt.Evtx('dump.evtx') as log: print(e_views.XML_HEADER); print('<Events>'); for record in log.records(): print(record.xml()); print('</Events>'); " > output.xml. The output shows the XML header and the start of the Events section. Then the user runs \$ ls, and the output shows dump.evtx and output.xml.

```
(alif@kali)-[~/Downloads/Owaps ctf week 3]
$ python3 -c "
import Evtx.Evtx as evt
import Evtx.Views as e_views

with evt.Evtx('dump.evtx') as log:
    print(e_views.XML_HEADER)
    print('<Events>')
    for record in log.records():
        print(record.xml())
    print('</Events>')
" > output.xml

(alif@kali)-[~/Downloads/Owaps ctf week 3]
$ ls
dump.evtx  output.xml
```

After running this script, output.xml was created and all further analysis was done by grepping through this file.

Investigation 1 - Identify the Stager File Name

Task: First, identify what was the stager file name?

Flag format: OWASPKL{xxxx.exe}

Step 1 - List all .exe files in the logs

First, I tried to list all .exe processes recorded in the logs.

There were too many normal Windows processes, so I needed to filter them.

cat output.xml | grep .exe

```
(alif@kali) ~/Downloads/Owaps ctf week 3
$ cat output.xml | grep .exe
<Data Name="Image">C:\Windows\Sysmon64.exe</Data>
<Data Name="CommandLine">C:\Windows\Sysmon64.exe</Data>
<Data Name="ParentImage">C:\Windows\System32\services.exe</Data>
<Data Name="ParentCommandLine">C:\Windows\system32\services.exe</Data>
<Data Name="Image">C:\Windows\System32\wbem\unsecapp.exe</Data>
<Data Name="CommandLine">C:\Windows\system32\wbem\unsecapp.exe -Embedding</Data>
<Data Name="ParentImage">C:\Windows\System32\svchost.exe</Data>
<Data Name="ParentCommandLine">C:\Windows\system32\svchost.exe -k DcomLaunch -p</Data>
<Data Name="Image">C:\Users\ligac\Downloads\Sysmon\Sysmon64.exe</Data>
<Data Name="Image">C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.28.240.0_x64__8wekyb3d8bbwe\WindowsPackageManagerServer.exe</Data>
<Data Name="CommandLine">C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.28.240.0_x64__8wekyb3d8bbwe\WindowsPackageManagerServer.exe -Embeddi
hg</Data>
<Data Name="ParentImage">C:\Windows\System32\svchost.exe</Data>
<Data Name="ParentCommandLine">C:\Windows\system32\svchost.exe -k DcomLaunch -p</Data>
<Data Name="Image">C:\Windows\System32\taskhostw.exe</Data>
<Data Name="OriginalFileName">taskhostw.exe</Data>
<Data Name="CommandLine">taskhostw.exe</Data>
<Data Name="ParentImage">C:\Windows\System32\svchost.exe</Data>
<Data Name="ParentCommandLine">C:\Windows\system32\svchost.exe -k netsvcs -p -s Schedule</Data>
<Data Name="Image">C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe</Data>
<Data Name="OriginalFileName">msedge.exe</Data>
<Data Name="CommandLine">C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe --type=utility --utility-sub-type=PooledProcess2 --lang=en-GB --serv
ice-sandbox-type=utility --ssd-no-pressure-read-main-dll --metrics-shmem-handle=8764,i,15503108158837571100,11922254670985591761,524288 --field-trial-handle
2160,i,10951072356334109064,1191380400481033935,262144 --variations-seed-version --pseudonymization-salt-handle=2164,i,5018163880319663648,1376660935717408
0296,4 --trace-process-track-uuid=3190709070645637904 --mojo-platform-channel-handle=8684 /prefetch:8</Data>
<Data Name="ParentImage">C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe</Data>
<Data Name="ParentCommandLine">C:\Program Files (x86)\Microsoft\Edge\Application\msedge.exe --profile-directory=Default</Data>
<Data Name="Image">C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.28.240.0_x64__8wekyb3d8bbwe\WindowsPackageManagerServer.exe</Data>
<Data Name="Image">C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.28.240.0_x64__8wekyb3d8bbwe\WindowsPackageManagerServer.exe -Embeddi
hg</Data>
<Data Name="ParentImage">C:\Windows\System32\svchost.exe</Data>
<Data Name="ParentCommandLine">C:\Windows\system32\svchost.exe -k DcomLaunch -p</Data>
<Data Name="Image">C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.28.240.0_x64__8wekyb3d8bbwe\WindowsPackageManagerServer.exe</Data>
<Data Name="CommandLine">C:\Program Files\WindowsApps\Microsoft.DesktopAppInstaller_1.28.240.0_x64__8wekyb3d8bbwe\WindowsPackageManagerServer.exe -Embeddi
```

Step 2 - Look for stager pattern

Since stagers often use PowerShell to hide their activity, I searched for PowerShell commands in the logs:

Cat output.xml | grep 'powershell'

```
(alif@kali) ~/Downloads/Owaps ctf week 3
$ cat output.xml | grep 'powershell'
<Data Name="Image">C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe</Data>
<Data Name="CommandLine">"powershell.exe" -WindowStyle Hidden -ExecutionPolicy Bypass -File C:\Windows\System32\ShowIP.ps1</Data>
<Data Name="Image">C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe</Data>
<Data Name="Image">C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe</Data>
<Data Name="CommandLine">"powershell.exe" -WindowStyle Hidden -ExecutionPolicy Bypass -File C:\Windows\System32\ShowIP.ps1</Data>
<Data Name="Image">C:\Windows\System32\WindowsPowerShell\v1.0\powershell.exe</Data>

(alif@kali) ~/Downloads/Owaps ctf week 3
$ grep -i "powershell" output.xml | less

(alif@kali) ~/Downloads/Owaps ctf week 3
$
```

This revealed a suspicious PowerShell command running hidden with execution policy bypassed which is a classic stager behaviour. To narrow things down further, I filtered out all normal Windows processes from CommandLine entries:

```
grep -i "CommandLine" output.xml | grep -v  
"svchost\powershell\edge\onedrive\Microsoft" | head -30
```

```
(alif@kali)-[~/Downloads/0waps ctf week 3]  
$ grep -i "CommandLine" output.xml | grep -v "svchost\powershell\edge\onedrive\Microsoft" | head -30  
<Data Name="CommandLine">C:\Windows\Sysmon64.exe</Data>  
<Data Name="ParentCommandLine">C:\Windows\system32\services.exe</Data>  
<Data Name="CommandLine">C:\Windows\system32\wbem\unsecapp.exe -Embedding</Data>  
<Data Name="CommandLine">taskhostw.exe</Data>  
<Data Name="ParentCommandLine">explorer.exe</Data>  
<Data Name="CommandLine">"C:\Windows\system32\notepad.exe" </Data>  
<Data Name="ParentCommandLine">explorer.exe</Data>  
<Data Name="CommandLine">phc.exe 10424</Data>  
<Data Name="ParentCommandLine">"C:\Windows\System32\cmd.exe" </Data>  
<Data Name="CommandLine">C:\Windows\system32\cmd.exe </Data>  
<Data Name="CommandLine">whoami /priv</Data>  
<Data Name="ParentCommandLine">C:\Windows\system32\cmd.exe </Data>  
<Data Name="CommandLine">netstat -a</Data>  
<Data Name="ParentCommandLine">C:\Windows\system32\cmd.exe </Data>  
<Data Name="CommandLine">route addr</Data>  
<Data Name="ParentCommandLine">C:\Windows\system32\cmd.exe </Data>  
<Data Name="CommandLine">route PRINT</Data>  
<Data Name="ParentCommandLine">C:\Windows\system32\cmd.exe </Data>  
<Data Name="CommandLine">consent.exe 6332 288 0000022B5C65D800</Data>  
<Data Name="CommandLine">"C:\Windows\system32\cmd.exe" </Data>  
<Data Name="ParentCommandLine">explorer.exe</Data>  
<Data Name="CommandLine">bcdedit /set testsigning on</Data>  
<Data Name="ParentCommandLine">"C:\Windows\system32\cmd.exe" </Data>  
<Data Name="CommandLine">bcdedit /set nointegritychecks on</Data>  
<Data Name="ParentCommandLine">"C:\Windows\system32\cmd.exe" </Data>  
<Data Name="CommandLine">"LogonUI.exe" /flags:0x4 /state0:0xa3148855 /state1:0x41c64e6d</Data>  
<Data Name="ParentCommandLine">winlogon.exe</Data>  
<Data Name="ParentCommandLine">C:\Windows\system32\services.exe</Data>  
<Data Name="ParentCommandLine">C:\Windows\system32\services.exe</Data>  
<Data Name="CommandLine">\SystemRoot\System32\smss.exe 00000084 00000088 </Data>  
  
(alif@kali)-[~/Downloads/0waps ctf week 3]  
$ grep -B 5 -A 15 "phc.exe" output.xml  
</System>
```

This revealed a suspicious executable: phc.exe. I then investigated it further:

grep -B 5 -A 15 "phc.exe" output.xml

```
(alif@kali)~/Downloads/Ownaps ctf week 3]
$ grep -B 5 -A 15 "phc.exe" output.xml
</System>
<EventData><Data Name="RuleName"></Data>
<Data Name="UtcTime">2026-06-05 04:30:32.762</Data>
<Data Name="ProcessGuid">{aa26e6de-50e8-6a22-6405-000000001000}</Data>
<Data Name="ProcessId">1136</Data>
<Data Name="Image">c:\Users\ligac\Downloads\phc.exe</Data>
<Data Name="FileVersion"></Data>
<Data Name="Description"></Data>
<Data Name="Product"></Data>
<Data Name="Company"></Data>
<Data Name="OriginalFileName"></Data>
<Data Name="CommandLine">phc.exe 10424</Data>
<Data Name="CurrentDirectory">c:\Users\ligac\Downloads</Data>
<Data Name="User">DESKTOP-LGT6HFQ\ligac</Data>
<Data Name="LogonGuid">{aa26e6de-45db-6a22-091d-250000000000}</Data>
<Data Name="LogonId">0</Data>
<Data Name="TerminalSessionId">1</Data>
<Data Name="IntegrityLevel">Medium</Data>
<Data Name="Hashes">MD5=8860ABA82B387B39E79A8C7FC43D42A6, SHA256=5D9F8486675F460BC85FD4323F80C58D0908F16510762F0647B4A36F11BDD78, IMPHASH=6D7793E48A21560E731D6B5AE1D0433A</Data>
<Data Name="ParentProcessGuid">{aa26e6de-4ab2-6a22-3504-000000001000}</Data>
<Data Name="ParentProcessId">7032</Data>
<Data Name="ParentImage">c:\Windows\System32\cmd.exe</Data>
<Data Name="ParentCommandLine">c:\Windows\System32\cmd.exe </Data>
<Data Name="ParentUser">DESKTOP-LGT6HFQ\ligac</Data>
</EventData>
</Event>
...
</System>
<EventData><Data Name="RuleName">DLL</Data>
<Data Name="UtcTime">2026-06-05 04:30:33.091</Data>
<Data Name="ProcessGuid">{aa26e6de-50e8-6a22-6405-000000001000}</Data>
<Data Name="ProcessId">1136</Data>
<Data Name="Image">c:\Users\ligac\Downloads\phc.exe</Data>
<Data Name="TargetFileName">c:\Users\ligac\AppData\Local\Microsoft\Windows\INetCache\IE\UAZCI1K7\maindll[1].dll</Data>
<Data Name="CreationUtcTime">2026-06-05 04:30:33.091</Data>
<Data Name="User">DESKTOP-LGT6HFQ\ligac</Data>
</EventData>
</Event>
...
<Event xmlns="http://schemas.microsoft.com/win/2004/08/events/event"><System><Provider Name="Microsoft-Windows-Sysmon" Guid="{5770385f-c22a-43e0-bf4c-06f5698ffbd9}"></Provider>
<EventID Qualifiers="">11</EventID>
<Version>2</Version>
<Level>4</Level>
<Task>11</Task>
<Opcode>0</Opcode>
<Keywords>0</Keywords>
<TimeCreated SystemTime="2026-06-05 04:30:33.093912+00:00"></TimeCreated>
<EventRecordID>29</EventRecordID>
...
</System>
<EventData><Data Name="RuleName">DLL</Data>
<Data Name="UtcTime">2026-06-05 04:30:33.093</Data>
```

What was found about phc.exe:

- Launched by cmd.exe
- Located in C:\Users\ligac\Downloads\ - not a normal location for software
- FileVersion, Description, Company fields are all blank (-) - not a legitimate program
- Downloaded maindll.dll to the INetCache folder
- Contacted suspicious domain: appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz
- Connected to attacker server: 104.21.54.36 on port 8443

These are textbook stager behaviours. I conclude that phc.exe is the stager.

Answer: OWASPKL {phc.exe}

Investigation 2 - Identify the Hijacked PID

Task: What was the PID that the threat actor hijacked?

Flag format: OWASPKL{xxxx}

From Investigation 1, I noticed that phc.exe was run with a number as its argument:

phc.exe 10424

This number looked like a Process ID (PID). I searched the logs to find what process was running with PID 10424:

grep -B 5 -A 15 "10424" output.xml | head -60

```
[~(otif@kali):~/Downloads/OWasp ctf week 3]
[~$ grep -B 5 -A 15 "10424" output.xml | head -60
<Data Name="FileVersion"></Data>
<Data Name="Description"></Data>
<Data Name="Product"></Data>
<Data Name="Company"></Data>
<Data Name="OriginalFileName"></Data>
<Data Name="CommandLine">phc.exe 10424</Data>
<Data Name="CurrentDirectory">C:\Users\ligac\Downloads</Data>
<Data Name="User">DESKTOP-LGT6HFQ\ligac</Data>
<Data Name="LogonGuid">{aa26e6de-45db-6a22-091d-250000000000}</Data>
<Data Name="LogonId">0</Data>
<Data Name="TerminalSessionId">1</Data>
<Data Name="IntegrityLevel">Medium</Data>
<Data Name="Hashes">MD5:88608A62B387B39E79A8C7FC43042A6_SHA256:5D9F8486675F460BC85FD4323F80C580D090BF16510762F06478A436F11BDD78_IMPHASH:6D7793E48A21560E731D6B5AE1D0433A</Data>
<Data Name="ParentProcessGuid">{aa26e6de-4ab2-6a22-3504-000000001000}</Data>
<Data Name="ParentProcessId">7032</Data>
<Data Name="ParentImage">C:\Windows\System32\cmd.exe</Data>
<Data Name="ParentCommandLine">C:\Windows\System32\cmd.exe </Data>
<Data Name="ParentUser">DESKTOP-LGT6HFQ\ligac</Data>
</EventData>
</Event>
--
<Security UserID="S-1-5-18"></Security>
</System>
<EventData><Data Name="RuleName"></Data>
<Data Name="UtcTime">2026-06-05 04:30:33.256</Data>
<Data Name="ProcessGuid">{aa26e6de-4f6b-6a22-1805-000000001000}</Data>
<Data Name="ProcessId">10424</Data>
<Data Name="QueryName">owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz</Data>
<Data Name="QueryStatus">0</Data>
<Data Name="QueryResults">::ffff:172.67.223.30;::ffff:104.21.54.36</Data>
<Data Name="Image">C:\Program Files\WindowsApps\Microsoft.MicrosoftOfficeHub_19.2605.59121.0_x64__8wekyb3d8bbwe\M365Copilot.exe</Data>
<Data Name="User">DESKTOP-LGT6HFQ\ligac</Data>
</EventData>
</Event>
--
<Event xmlns="http://schemas.microsoft.com/win/2004/08/events/event"><System><Provider Name="Microsoft-Windows-Sysmon" Guid="{5770385f-c22a-43e0-bf4c-06f5698ffbd9}"></Provider>
<EventID Qualifiers="0">1</EventID>
<Version>5</Version>
<Level>4</Level>
<Task>2</Task>
<Opcode>0</Opcode>
<Keywords>0</Keywords>
--
<Security UserID="S-1-5-18"></Security>
</System>
<EventData><Data Name="RuleName"></Data>
<Data Name="UtcTime">2026-06-05 04:30:39.015</Data>
<Data Name="ProcessGuid">{aa26e6de-4f6b-6a22-1805-000000001000}</Data>
<Data Name="ProcessId">10424</Data>
<Data Name="QueryName">owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz</Data>
<Data Name="QueryStatus">0</Data>
<Data Name="QueryResults">::ffff:104.21.54.36;::ffff:172.67.223.30</Data>
<Data Name="Image">C:\Program Files\WindowsApps\Microsoft.MicrosoftOfficeHub_19.2605.59121.0_x64__8wekyb3d8bbwe\M365Copilot.exe</Data>
<Data Name="User">DESKTOP-LGT6HFQ\ligac</Data>
</EventData>
</Event>
```

The results showed that PID 10424 belonged to M365Copilot.exe, and it was making DNS queries to owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz, a suspicious domain that a legitimate Microsoft Office app should never contact.

Answer: OWASPKL{10424}

Investigation 3 - Identify the Hijacked Process Name

Task: What was the name of the process that was hijacked?

Flag format: OWASPKL{process.exe}

From Investigation 2, the process with PID 10424 was already identified. The full image path from the logs was:

C:\Program
Files\WindowsApps\Microsoft.MicrosoftOfficeHub_19.2605.59121.0_x64__8wekyb3d8
bbwe\M365Copilot.exe

M365Copilot.exe is a legitimate Microsoft app that was hijacked by phc.exe to use as a disguise for C2 communication. This is a process hijacking technique, malware injects itself into a trusted process so its network traffic looks legitimate.

Answer: OWASPKL{M365Copilot.exe}

Task: Identify the URL that the stager retrieves the payload from.
Flag format: OWASPKL{URL}

From Investigation 1, I already knew phc.exe contacted this :
appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz

```
grep -i "NetworkConnect\|DestinationIp\|DestinationPort" output.xml | head -30
```

This revealed port 8443 was used which is an unusual port. All other connections used port 443 (standard HTTPS).

Step 3 - Find the filename

From the file creation events, the downloaded file was named:

maindll[1].dll → maindll.dll

The [1] suffix means Windows downloaded this file from the internet via the INetCache mechanism.

Step 4 - Confirm with curl

I tried to access the URL directly to confirm. First attempt with HTTP returned an error saying the server requires HTTPS. Then with HTTPS:

curl -vk https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll

```
(alif@kali) [~/Downloads/0waps ctf week 3]
$ curl -vk https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll
* Host appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443 was resolved.
* IPv6: (none)
* IPv4: 104.21.54.36
* Trying 104.21.54.36:8443...
* ALPN: curl offers h2,http/1.1
* TLSv1.3 (OUT), TLS handshake, Client hello (1):
* SSL Trust: peer verification disabled
* TLSv1.3 (IN), TLS handshake, Server hello (2):
* TLSv1.3 (IN), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (IN), TLS handshake, Encrypted Extensions (8):
* TLSv1.3 (IN), TLS handshake, Certificate (11):
* TLSv1.3 (IN), TLS handshake, CERT verify (15):
* TLSv1.3 (IN), TLS handshake, Finished (20):
* TLSv1.3 (OUT), TLS change cipher, Change cipher spec (1):
* TLSv1.3 (OUT), TLS handshake, Finished (20):
* SSL connection using TLSv1.3 / TLS_AES_256_GCM_SHA384 / X25519MLKEM768 / id-ecPublicKey
* ALPN: server accepted h2
* Server certificate:
* subject: CN=3cc83feaa3b37384a190dd84b25a4592.xyz
* start date: Jun  3 10:17:10 2026 GMT
* expire date: Sep  1 11:14:35 2026 GMT
* issuer: C=US; O=Google Trust Services; CN=WE1
* Certificate level 0: Public key type EC/prime256v1 (256/128 Bits/secBits), signed using ecdsa-with-SHA256
* Certificate level 1: Public key type EC/prime256v1 (256/128 Bits/secBits), signed using ecdsa-with-SHA384
* Certificate level 2: Public key type EC/secp384r1 (384/192 Bits/secBits), signed using sha256WithRSAEncryption
* OpenSSL verify result: 14
* SSL certificate verification failed, continuing anyway!
* Established connection to appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz (104.21.54.36 port 8443) from 192.168.180.130 port 46442
* using HTTP/2
* [HTTP/2] [1] OPENED stream for https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll
* [HTTP/2] [1] [:method: GET]
* [HTTP/2] [1] [:scheme: https]
* [HTTP/2] [1] [:authority: appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443]
* [HTTP/2] [1] [:path: /maindll.dll]
* [HTTP/2] [1] [user-agent: curl/8.19.0]
* [HTTP/2] [1] [accept: /*/*]
> GET /maindll.dll HTTP/2
> Host: appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443
> User-Agent: curl/8.19.0
> Accept: /*/*
>
* Request completely sent off
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
* TLSv1.3 (IN), TLS handshake, Newsession Ticket (4):
< HTTP/2 200
< date: Sat, 06 Jun 2026 15:33:48 GMT
< content-type: application/octet-stream
< content-length: 107520
< server: cloudflare
< last-modified: Fri, 05 Jun 2026 04:03:10 GMT
< nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}
< etag: "6a224a7e-1a400"
< accept-ranges: bytes
```

The server responded with HTTP 200 and the file was accessible. URL confirmed.

Answer:

OWASP{https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll}

Investigation 5 - Identify Valid C2 Commands

Task: Identify the valid command(s) that the C2 beacon ran on the machine.

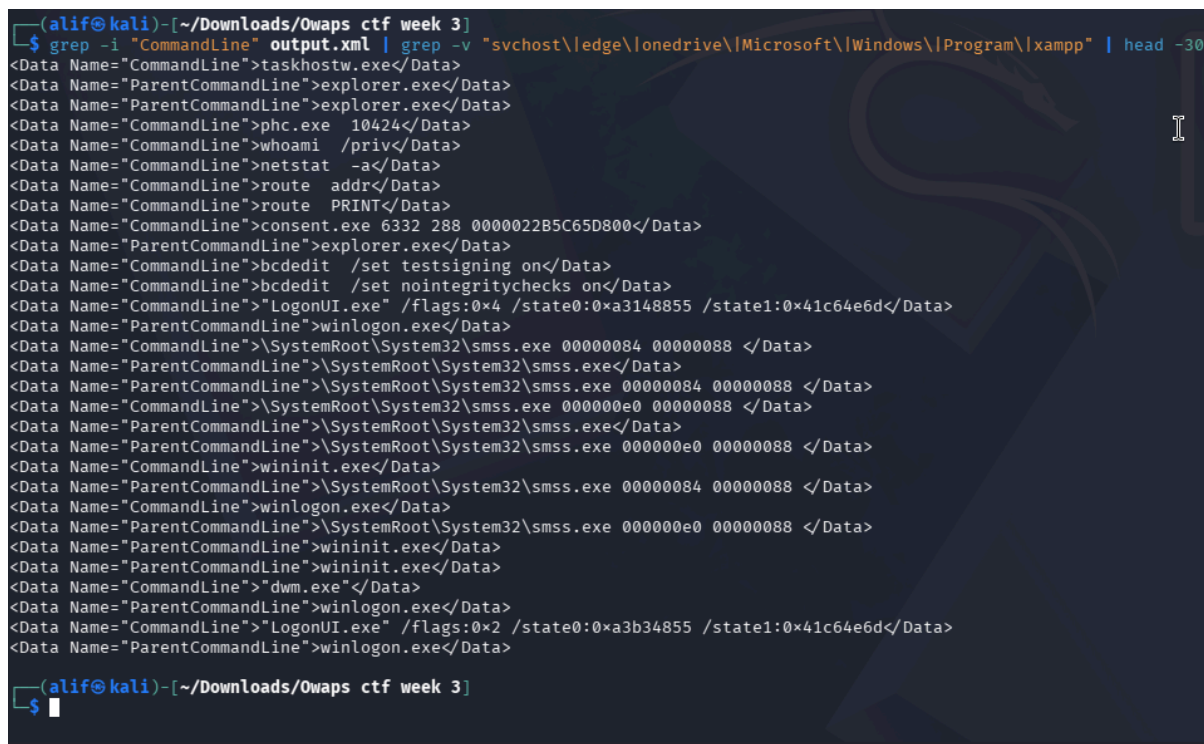
Flag format: OWASPKL{command}

A C2 beacon is a regular check-in signal that malware sends to the attacker's server. The attacker can then send commands back to execute on the victim machine.

Step 1 - Filter for suspicious commands

I filtered out all normal Windows processes to find unusual commands:

```
grep -i "CommandLine" output.xml | grep -v "svchost\|edge\|onedrive\|Microsoft\|Windows\|Program\|xampp" | head -30
```



```
(alif@kali) - [~/Downloads/0waps ctf week 3]
$ grep -i "CommandLine" output.xml | grep -v "svchost\|edge\|onedrive\|Microsoft\|Windows\|Program\|xampp" | head -30
<Data Name="CommandLine">taskhostw.exe</Data>
<Data Name="ParentCommandLine">explorer.exe</Data>
<Data Name="ParentCommandLine">explorer.exe</Data>
<Data Name="CommandLine">phc.exe 10424</Data>
<Data Name="CommandLine">whoami /priv</Data>
<Data Name="CommandLine">netstat -a</Data>
<Data Name="CommandLine">route addr</Data>
<Data Name="CommandLine">route PRINT</Data>
<Data Name="CommandLine">consent.exe 6332 288 0000022B5C65D800</Data>
<Data Name="ParentCommandLine">explorer.exe</Data>
<Data Name="CommandLine">bcdedit /set testsigning on</Data>
<Data Name="CommandLine">bcdedit /set nointegritychecks on</Data>
<Data Name="CommandLine">"LogonUI.exe" /flags:0x4 /state0:0xa3148855 /state1:0x41c64e6d</Data>
<Data Name="ParentCommandLine">winlogon.exe</Data>
<Data Name="CommandLine">"\SystemRoot\System32\smss.exe 00000084 00000088 </Data>
<Data Name="ParentCommandLine">"\SystemRoot\System32\smss.exe 00000084 00000088 </Data>
<Data Name="CommandLine">"\SystemRoot\System32\smss.exe 000000e0 00000088 </Data>
<Data Name="ParentCommandLine">"\SystemRoot\System32\smss.exe 000000e0 00000088 </Data>
<Data Name="CommandLine">wininit.exe</Data>
<Data Name="ParentCommandLine">"\SystemRoot\System32\smss.exe 00000084 00000088 </Data>
<Data Name="CommandLine">winlogon.exe</Data>
<Data Name="ParentCommandLine">"\SystemRoot\System32\smss.exe 000000e0 00000088 </Data>
<Data Name="ParentCommandLine">wininit.exe</Data>
<Data Name="ParentCommandLine">wininit.exe</Data>
<Data Name="CommandLine">"dwm.exe"</Data>
<Data Name="ParentCommandLine">winlogon.exe</Data>
<Data Name="CommandLine">"LogonUI.exe" /flags:0x2 /state0:0xa3b34855 /state1:0x41c64e6d</Data>
<Data Name="ParentCommandLine">winlogon.exe</Data>

(alif@kali) - [~/Downloads/0waps ctf week 3]
$
```

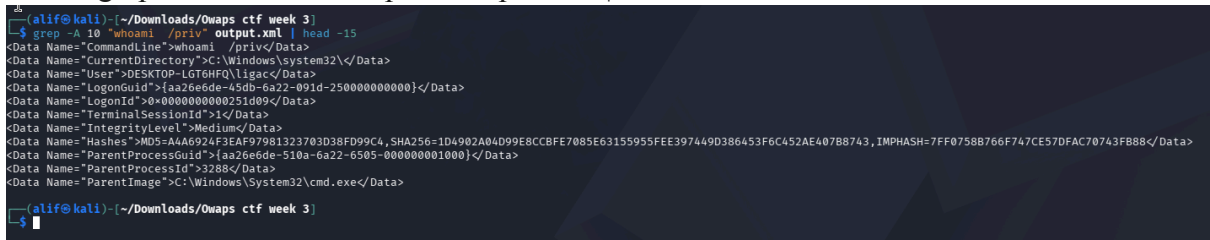
This revealed several suspicious commands with double spaces between the command and argument — a sign they were sent remotely by the C2 beacon rather than typed manually:

```
whoami /priv
netstat -a
route addr
route PRINT
bcdedit /set testsigning on
bcdedit /set nointegritychecks on
```

Step 2 - Trace commands back to the C2 process

I traced whoami /priv back to its parent process:

```
grep -A 10 "whoami /priv" output.xml | head -15
```



The parent was cmd.exe with PID 3288. I then traced PID 3288:

```
grep -B 5 -A 15 ">3288" output.xml | head -30
```



PID 3288 had ParentProcessGuid matching M365Copilot.exe (PID 10424) — the hijacked C2 process. This confirmed all these commands were sent by the C2 beacon.

Step 3 - Determine which commands are valid

Not all commands would have succeeded. The user was running at Medium integrity level, so commands requiring admin (like bcdedit) would fail. route addr is also invalid syntax.

Field	Value
whoami /priv	Valid — always works
netstat -a	Valid — always works
route PRINT	Valid — always works
route addr	Invalid — wrong syntax
bcdedit /set testsigning on	Invalid — needs admin
bcdedit /set nointegritychecks on	Invalid — needs admin

Answer: OWASPKL{whoami /priv} | OWASPKL{netstat -a} | OWASPKL{route PRINT}

Investigation 6 - Identify the Payload Location

Task: Identify the payload location in the machine.

Flag format: OWASPKL{path\path....\path\payload}

From Investigation 1, I already knew phc.exe was downloading a file. I searched for files created in temp/appdata locations:

```
grep -i "temp\appdata\|%temp%" output.xml | head -30
```

This showed two locations for the same file:

- INetCache\IE\UAZCI1K7\maindll[1].dll - temporary internet cache (downloaded from web)
- AppData\Local\Temp\maindll.dll - the final payload location

The stager downloaded the payload to the internet cache first, then copied it to the Temp folder. The Temp folder is the actual payload location because it is where the malware was placed for execution. Malware commonly uses the Temp folder since any user can write to it and it is sometimes overlooked by antivirus.

Answer: OWASPKL{C:\Users\ligac\AppData\Local\Temp\maindll.dll}

Investigation 7 - Identify the C2 Server URL

Task: Identify the C2 Server URL.

Flag format: OWASPKL{URL}

From previous investigations, two suspicious domains were found. I listed all unique DNS queries from the logs:

```
grep -i "QueryName" output.xml | sort -u
```

Two attacker-controlled domains were identified:

Field	Value
appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz	Used by phc.exe — payload delivery
owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz	Used by M365Copilot.exe — C2 beaconing

The owaspkl domain is the C2 server because it was used by the hijacked M365Copilot process to repeatedly check in with the attacker. I verified the server was alive:

```
curl -vk https://owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz:8443
```

```
< !DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
<style>
html { color-scheme: light dark; }
body { width: 35em; margin: 0 auto;
font-family: Tahoma, Verdana, Arial, sans-serif; }
</style>
</head>
<body>
<h1>Welcome to nginx!</h1>
<p>If you see this page, the nginx web server is successfully installed and
working. Further configuration is required.</p>

<p>For online documentation and support please refer to
<a href="http://nginx.org/">nginx.org</a>.<br/>
Commercial support is available at
<a href="http://nginx.com/">nginx.com</a>.</p>

<p><em>Thank you for using nginx.</em></p>
</body>
</html>
* Connection #0 to host owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz:8443 left intact
```

The server responded with HTTP 200 and showed a default nginx page, confirming it was active.

Answer: OWASPKL{https://owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz}

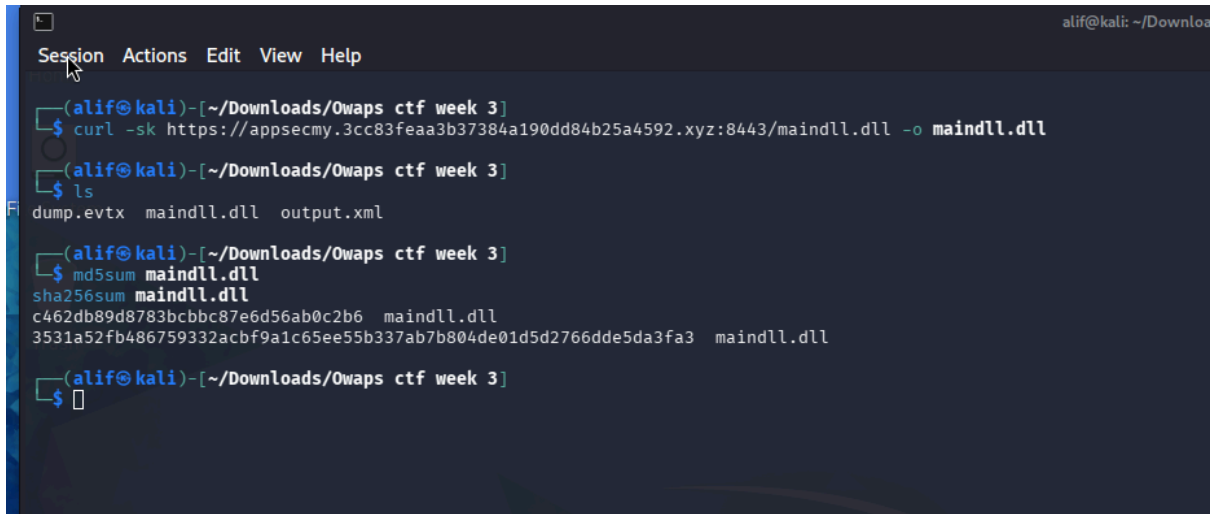
Investigation 8 - Identify the Payload Hash

Task: What is the hash of the actual payload?

Flag format: OWASPKL{filehash}

The payload file maindll.dll was not directly logged with its hash in the Sysmon logs. Since I already knew the download URL from Investigation 4, I downloaded the file and calculated the hash myself:

```
curl -sk https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll -o maindll.dll
md5sum maindll.dll
sha256sum maindll.dll
```



```
alif@kali: ~/Downloads
Session Actions Edit View Help
(alif@kali)-[~/Downloads/Owaps ctf week 3]
$ curl -sk https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll -o maindll.dll
(alif@kali)-[~/Downloads/Owaps ctf week 3]
$ ls
dump.evtx  maindll.dll  output.xml
(alif@kali)-[~/Downloads/Owaps ctf week 3]
$ md5sum maindll.dll
c462db89d8783bcbbc87e6d56ab0c2b6  maindll.dll
$ sha256sum maindll.dll
3531a52fb486759332acbf9a1c65ee55b337ab7b804de01d5d2766dde5da3fa3  maindll.dll
(alif@kali)-[~/Downloads/Owaps ctf week 3]
$
```

Both MD5 and SHA256 hashes were generated. After trying both, the SHA256 hash was the correct one accepted by the CTF.

Field	Value
MD5	c462db89d8783bcbbc87e6d56ab0c2b6
SHA256	3531a52fb486759332acbf9a1c65ee55b337ab7b804de01d5d2766dde5da3fa3

Answer:

OWASPKL{3531a52fb486759332acbf9a1c65ee55b337ab7b804de01d5d2766dde5da3fa3}

Investigation 9 - Identify the Root C2 Server IP

Task: Identify the root server IP address of the C2.

Note: This challenge moves towards Threat Hunting / Intelligence.

Step 1 - IPs from Sysmon logs

The IPs found in the logs were:

- 104.21.54.36 - Cloudflare proxy
- 172.67.223.30 - Cloudflare proxy

Both IPs belonged to Cloudflare (AS 13335), meaning the real server was hidden behind Cloudflare's proxy. These are not the origin server.

Step 2 - Threat Intelligence with VirusTotal

I uploaded the payload hash (maindll.dll) to VirusTotal and checked the Relations tab. This revealed a third IP that was not in the Sysmon logs:

- 172.178.240.163 - Microsoft AS8075

DETECTION	DETAILS	RELATIONS	BEHAVIOR	COMMUNITY
Contacted URLs (1) ⓘ				
Scanned	Detections	Status	URL	
2026-06-05	2 / 91	522	https://owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz/64sx8nsQmuQ9-Dz6V9p0jQWLKzXlm9rZ9Lh1st70NJBIR0Cj3wTu4y8nGXDFx-NPReIT0L_MsEIM3ME7rTzAyxmB4X93CFM5VCWn	
Contacted Domains (2) ⓘ				
Domain	Detections	Created	Registrar	
3cc83feaa3b37384a190dd84b25a4592.xyz	0 / 91	-	-	
owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz	0 / 91	-	-	
Contacted IP addresses (3) ⓘ				
IP	Detections	Autonomous System	Country	
104.21.54.36	0 / 91	13335	-	
172.178.240.163	0 / 91	8075	US	
172.67.223.30	0 / 91	13335	-	
Bundled Files (6) ⓘ				
Scanned	Detections	File type	Name	
2026-06-06	0 / 60	ISO image	.fptable	
?	?	file	.data	
?	?	file	.text	
?	?	file	.reloc	
?	?	file	.rdata	
?	?	file	.pdata	

I investigated this IP using curl:

```
curl -vk https://172.178.240.163
```

The certificate showed CN=nw-umwatson.events.data.microsoft.com — this is a legitimate Microsoft telemetry server, not the C2. Dead end.

Step 3 - Finding the Origin IP with Censys

Since all known IPs were either Cloudflare or Microsoft, I used Censys to search for the real server by its SSL certificate domain pattern:

<https://search.censys.io/search?resource=hosts&q=3cc83feaa3b37384a190dd84b25a4592.xyz>

 **43.216.92.187** • HOST
ec2-43-216-92-187.ap-southeast-5.compute.amazonaws.com

DEFAULT_LANDING_PAGE

Network (AS)

AMAZON-02 - Amazon.com, Inc. (16509)

Location

Kuala Lumpur (MY) 

Services (1)

8443 / HTTP

Software (1)

F5 Nginx

MATCHED FIELDS

host.services.cert.names

kdjebat.3cc83feaa3b37384a190dd84...

8443 / HTTP

host.services.cert.parsed.subject.common_name

kdjebat.3cc83feaa3b37384a190dd84...

8443 / HTTP

host.services.cert.parsed.subject_dn

N=kdjebat.3cc83feaa3b37384a190dd...

8443 / HTTP

Censys found a host that matched the attacker infrastructure. The result was confirmed as the real origin server by cross-referencing several pieces of evidence collected throughout the investigation. First, the server was running on port 8443 which matched exactly what was seen in the Sysmon logs when phc.exe made its network connection. Second, the server software was identified as Nginx, the same web server that responded when the C2 domain was curled earlier. Third, the SSL certificate on the server used the same domain pattern *.3cc83feaa3b37384a190dd84b25a4592.xyz, confirming it belongs to the same attacker infrastructure. Finally, the server was located in Kuala Lumpur, Malaysia which is consistent with the CTF context, OWASPKL stands for OWASP Kuala Lumpur. All four pieces of evidence together confirmed that 43.216.92.187 is the real C2 server hiding behind Cloudflare.

Field	Value
IP	43.216.92.187
Hostname	ec2-43-216-92-187.ap-southeast-5.compute.amazonaws.com
Network	Amazon AWS (AS16509)
Location	Kuala Lumpur, Malaysia
Port	8443
Software	F5 Nginx
Certificate	kdjebat.3cc83feaa3b37384a190dd84b25a4592.xyz

Answer: OWASPKL{43.216.92.187}

Summary - Full Attack Chain

Here is the complete picture of what happened on the victim machine:

```
cmd.exe
|
+-- phc.exe 10424 (stager, run from Downloads folder)
|
+-- Downloads maindll.dll from:
|   https://appsecmy.xxx.xyz:8443/maindll.dll
|
+-- Injects into M365Copilot.exe (PID 10424)
|
+-- Beacons to: owaspkl.xxx.xyz
|   (Real server: 43.216.92.187, AWS KL)
|
+-- Spawns cmd.exe
|
+-- whoami /priv
+-- netstat -a
+-- route PRINT
```

Field	Value
Stager	phc.exe
Hijacked Process	M365Copilot.exe (PID 10424)
Payload	maindll.dll
Payload URL	https://appsecmy.3cc83feaa3b37384a190dd84b25a4592.xyz:8443/maindll.dll
Payload Location	C:\Users\ligac\AppData\Local\Temp\maindll.dll
Payload Hash (SHA256)	3531a52fb486759332acbf9a1c65ee55b337ab7b804de01d5d2766dde5da3fa3
C2 Domain	owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz
C2 URL	https://owaspkl.3cc83feaa3b37384a190dd84b25a4592.xyz
C2 Root IP	43.216.92.187 (AWS, Kuala Lumpur)
C2 Port	8443
C2 Commands	whoami /priv, netstat -a, route PRINT